P0634R1
2017-10-15

Nina Ranns (dinka.ranns@gmail.com)
Daveed Vandevoorde (daveed@edg.com)

Core Working Group (CWG)

Down with `typename`!

Summary

If `X::Y` — where `T` is a template parameter — is to denote a type, it must be preceded by the keyword `typename`; otherwise, it is assumed to denote a name producing an expression. There are currently two notable exceptions to this rule: *base-specifier*s and *mem-initializer-id*s. For example:

```
template<class T>
struct D: T::B { // No typename required here.
};
```

Clearly, no `typename` is needed for this *base-specifier* because nothing but a type is possible in that context. However, there are several other places where we know only a type is possible and asking programmers to nonetheless specify the *typename* keyword feels like a waste of source code space (and is detrimental to readability).

We therefore propose we make `typename` optional in a number of commonplace contexts that are known to only permit type names.

A cursory read through some common standard library headers suggests that by-far most occurrences of `typename` for the purpose of disambiguating type names from other names can be eliminated with the new rules.

The EDG front end has an `implicit typename` mode to emulate pre-C++98 compilers that didn't parse templates in their generic form. Although that mode doesn't exactly cover the contexts where we are proposing to make `typename` optional, the implementation effort is similar (and not excessively expensive).

This proposal was approved in Toronto (July 2017) by EWG:

SF: 14 | F: 8 | N: 1 | A: 0 | SA: 0

Wording changes

☐Hide deleted text

Change in [temp.res] paragraph 3:

When a *qualified-id* is intended to refer to a type that is not a member of the current instantiation (17.7.2.1) and its *nested-name-specifier* refers to a dependent type, it shall be prefixed by the keyword `typename`, ~~forming a *typename-specifier*. If the *qualified-id* in a *typename-specifier* does not denote a type or a class template, the program is ill-formed.~~unless:

- it is a qualified name in an *type-id-only context* (see below), or
- it is the *decl-specifier-seq* of a:
  - *simple-declaration* in namespace scope,
  - *member-declaration* in class scope,
  - *parameter-declaration* in a function-declarator for a function, pointer to function, reference to function, or pointer-to-member function at class sope [*Note*: This includes friend function declarations. —*end note*],
  - *parameter-declaration* of a function parameter appearing in a *lambda-declarator*, or
  - *parameter-declaration* in a (non-type) *template-parameter*.

A qualified name is said to be in an *type-id-only context* if it appears in a *type-id*, *new-type-id*, or *defining-type-id* and the smallest enclosing *type-id*, *new-type-id*, or *defining-type-id* is a

- *new-type-id*,
- *defining-type-id*,

- *trailing-return-type*,
- default argument of a *type-parameter* of a template, or
- *type-id* of a `static_cast`, `const_cast`, `reinterpret_cast`, or `dynamic_cast`.

[*Example:*

```
template<class T> T::R f(T::P); // OK, global scope
template<class T> struct S {
   using Ptr = PtrTraits::Ptr; // OK, in a defining-type-id
   T::R f(T::P p) { // OK
      return static_cast<T::R> // OK, type-id of a static_cast
   }
   auto g() -> S<T*>::Ptr;     // OK, trailing-return-type
};
template<typename T> void f() {
   void (*pf)(T::X);           // Variable pf of type void* initialized with T::X
   void g(T::X);               // Error: T::X (not at class scope) does not denote a type
};
```

— end example]

A qualified name prefixed with `typename` in this way forms a *typename-specifier:*

typename-specifier:
`typename` nested-name-specifier identifier
`typename` nested-name-specifier $\textit{template}_{opt}$ *simple-template-id*

If the qualified-id in a typename-specifier does not denote a type or a class template, the program is ill-formed.